

# Eval-Ai

## Eval AI Agent - 24 Task Quick Reference

Hands-on exercises for evaluating AI coding agents.  
Covers beginner, intermediate, advanced, and DevOps tracks.

### Key constants

Eval API URL: `http://127.0.0.1:8788`

Start server: `make eval-api`

Dashboard: `http://127.0.0.1:8788/`

Task guide: `GET /api/agent/guide/{ID}`

Submit work: `POST /api/agent/submit`

Verdicts: `ok | partial | mismatch`

Task IDs: `B1-B6, I1-I6, A1-A6, D1-D6`

## How to Use the Eval API

---

1. Clone repo and run: `make setup && make eval-api`
2. Open dashboard: `http://127.0.0.1:8788`
3. Get task instructions: `curl http://127.0.0.1:8788/api/agent/guide/B4`
4. Do work in the task folder (e.g. `beginner/B4-fastapi-service/`)
5. Submit: `POST /api/agent/submit` with `task_id` and `output_path`
6. Check verdict on dashboard or `GET /api/portfolio`

### Main API Endpoints

- `GET /` Live dashboard (browser)
- `GET /api/health` Server health check
- `GET /api/docs` Full API list
- `GET /api/tasks` All 24 tasks
- `GET /api/tasks/{id}` One task status
- `GET /api/agent/guide/{id}` Task instructions (read first)
- `POST /api/agent/submit` Submit your work for checking
- `GET /api/portfolio` All tasks + results (JSON)
- `POST /api/portfolio/refresh` Re-scan repo files

### Submit Example

```
curl -X POST http://127.0.0.1:8788/api/agent/submit \
  -H "Content-Type: application/json" \
  -d '{"task_id": "B2", "output_path": "beginner/B2-api-endpoint-map/API_MAP.md"}'
```

## Beginner (B1-B6)

---

### B1 - Repo Inventory

Purpose: Scan repo and list files by category

Output: REPORT.md, inventory.csv

Check: Match paths/counts in CSV

### B2 - API Map

Purpose: Find all HTTP routes in the project

Output: API\_MAP.md, endpoints.csv

Check: 25 routes across 6 services

Needs: B1

### B3 - Test Discovery

Purpose: Find all tests and run them

Output: TEST\_REPORT.md

Check: make test

### B4 - FastAPI Service

Purpose: Build transaction REST API in Python

Output: app/main.py, tests

Check: pytest (port 8000)

API: GET/POST /transactions, GET /balance, GET /health

### B5 - Node API

Purpose: Same transaction API in Express

Output: src/app.js, tests

Check: npm test (port 3001)

Needs: B4

API: Same routes as B4 (no get-by-id)

### B6 - Rust CLI

Purpose: Log analyzer command-line tool

Output: src/main.rs, tests

Check: cargo test

API: No HTTP - CLI only

## Intermediate (I1-I6)

---

### I1 - ER Diagram

Purpose: Draw database entities and relationships

Output: ER\_REPORT.md, er-diagram.mmd

Check: Match B4/D2/A3 models

Needs: B1

### I2 - End-to-End Trace

Purpose: Trace request flow through system

Output: FLOW\_TRACE.md, sequence-diagram.mmd

Check: Follow steps in trace doc

Needs: B2

### I3 - Safe Change

Purpose: Plan a small code change with risk report

Output: CHANGE\_REPORT.md, risk-assessment.md

Check: Risk scope matches report

Needs: B3

### I4 - FastAPI + Node Pair

Purpose: Currency API + Node client

Output: fastapi main.py, node cli.js

Check: pytest + npm test (port 8000)

Needs: B4, B5

API: POST /convert, GET /health

### I5 - Dockerize

Purpose: Put a service in a Docker container

Output: Dockerfile, DOCKER\_REPORT.md

Check: verify-docker.sh

### I6 - Bug Diagnosis

Purpose: Find root cause and document fix

Output: BUG\_REPORT.md, ROOT\_CAUSE\_ANALYSIS.md

Check: Match RCA structure

Needs: I2

## Advanced (A1-A6)

---

### A1 - Parallel Planning

Purpose: Plan work for multiple agents/branches

Output: PARALLEL\_EXECUTION\_PLAN.md, agent-prompts.md

Check: Lane prompts match plan

### A2 - Parallel Worktrees

Purpose: Use git worktrees for parallel dev

Output: WORKTREE\_EXECUTION\_REPORT.md, merge-log.md

Check: Worktree workflow in report

Needs: A1

### A3 - Polyglot System

Purpose: FastAPI -> queue -> Node worker -> Rust

Output: Python + Node + Rust services

Check: integration-test.sh (port 8000)

Needs: B4, B5, B6, I4

API: POST /transactions returns 202

### A4 - Modernization

Purpose: Find what to upgrade in codebase

Output: MODERNIZATION\_REPORT.md, PRIORITIZATION\_MATRIX.md

Check: Priority matrix is reference

Needs: B1, B3

### A5 - Agent Review

Purpose: Review code/agent output with findings

Output: REVIEW\_REPORT.md, FINDINGS\_MATRIX.md

Check: Findings categorized like matrix

### A6 - Performance

Purpose: Benchmark and optimize code

Output: PERFORMANCE\_REPORT.md, run-benchmark.sh

Check: Run benchmark script

## DevOps (D1-D6)

---

### D1 - Terraform

Purpose: AWS infra as code (Lambda, etc.)

Output: main.tf, lambda/index.py

Check: terraform validate

### D2 - Docker Compose

Purpose: Multi-service stack (API + Postgres)

Output: docker-compose.yml, e2e\_test.sh

Check: API on port 8200

Needs: I5

API: GET/POST /transactions, GET /health

### D3 - CI Pipeline

Purpose: Local CI (lint, test, build)

Output: PIPELINE\_REPORT.md, run-pipeline-local.sh

Check: Run local pipeline script

Needs: B3, I5

### D4 - Kubernetes

Purpose: K8s deployment manifests

Output: k8s/deployment.yaml, validate-manifests.sh

Check: Manifest validation passes

Needs: I5

### D5 - Dev Environment

Purpose: One-command setup for whole repo

Output: README.md, bootstrap docs

Check: make bootstrap / make test

Needs: B3

### D6 - Observability

Purpose: Logs + Prometheus + Grafana

Output: docker-compose.yml, service main.py

Check: GET /metrics (port 8000/18080)

Needs: I5

API: B4 routes + GET /metrics

## One-Line Summary (All 24)

---

- B1: List everything in the repo
- B2: Map all API endpoints
- B3: Find and run all tests
- B4: Build FastAPI transaction API
- B5: Build same API in Node.js
- B6: Build Rust log analyzer CLI
- I1: Draw ER diagram for data model
- I2: Trace request flow end-to-end
- I3: Plan a safe code change
- I4: Currency API + Node client
- I5: Dockerize a service
- I6: Diagnose and document a bug
- A1: Plan parallel agent work
- A2: Use git worktrees in parallel
- A3: Connect Python + Node + Rust system
- A4: Plan codebase modernization
- A5: Review agent/code quality
- A6: Benchmark and optimize performance
- D1: Terraform AWS infra
- D2: Docker Compose multi-service stack
- D3: CI pipeline (lint/test/build)
- D4: Kubernetes manifests
- D5: Reproducible dev environment setup
- D6: Monitoring (metrics + Grafana)

## Suggested Pipeline Order

---

B1 -> B2 -> B3 -> B4 -> B5 -> B6 -> I1 -> I2 -> I3 -> I4 -> I5 -> I6 -> A1 -> A2 -> A3 -> A4 -> A5 -> A6 -> D1 -> D2 -> D3 -> D4 -> D5 -> D6

## Task Types

---

- analysis - B1, B2, B3 (read repo, write reports)
- code - B4, B5, B6, I4, A3, A6 (write and run code)
- report - I1, I2, I3, I6, A1, A2, A4, A5 (docs and diagrams)
- infra - I5, D1-D6 (Docker, Terraform, CI, K8s)